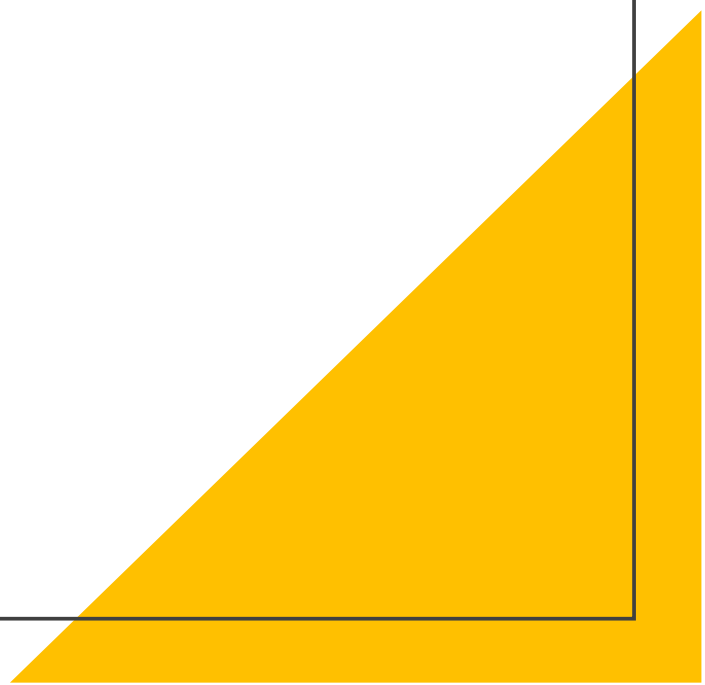


RxJS



RxJS

RxJS 是一個使用可觀察序列編寫非同步和基於事件的事件的函式庫。它提供了一種核心型別，即 Observable、一些周邊型別

（Observer、Scheduler、Subjects）以便將非同步事件作為集合進行處理。

可以將 RxJS 視為處理事件的 Lodash(模組化)。

可觀察者（Observables）

Observable 是個多值的惰性 Push 集合。

左圖是一個 Observable，它在訂閱時立即（同步）推送值 1、2、3，並且在 subscribe 呼叫過一秒之後推送值 4，然後完成，執行結果如下。

```
just before subscribe
got value 1
got value 2
got value 3
just after subscribe
got value 4
done
```

```
import { Observable } from 'rxjs';

const observable = new Observable((subscriber) => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.next(3);
  setTimeout(() => {
    subscriber.next(4);
    subscriber.complete();
  }, 1000);
});

console.log('just before subscribe');
observable.subscribe({
  next(x) {
    console.log('got value ' + x);
  },
  error(err) {
    console.error('something wrong occurred: ' + err);
  },
  complete() {
    console.log('done');
  },
});
console.log('just after subscribe');
```

觀察者 (Observer)

Observer 是 Observable 傳遞的各個值的消費者。Observer 只是一組回呼，對應於 Observable 傳遞的每種型別的通知：next、error 和 complete。要使用 Observer，請將其提供給 Observable 的 subscribe。

```
const observer = {  
  next: (x) => console.log('Observer got a next value: ' + x),  
  error: (err) => console.error('Observer got an error: ' + err),  
  complete: () => console.log('Observer got a complete notification'),  
};
```

運算子 (Operators)

儘管 Observable 是基礎，但 RxJS 的運算子是最有用的。運算子是能讓你以宣告方式輕鬆組合複雜非同步程式碼的基本構造塊。

運算子是函式。有兩種運算子：

- 可聯入通道的運算子
- 建立運算子

運算子 (Operators)

可聯入通道的運算子是可以使用語法 `observableInstance.pipe(operator())` 聯入 Observables 通道的型別。其中包括 `filter(...)` 和 `mergeMap(...)`。呼叫時，它們不會更改現有的 Observable 實例。相反，它們回傳一個新的 Observable，其訂閱邏輯是基於第一個 Observable 的。

```
import { of, map } from 'rxjs';

of(1, 2, 3)
  .pipe(map((x) => x * x))
  .subscribe((v) => console.log(`value: ${v}`));

// Logs:
// value: 1
// value: 4
// value: 9
```

運算子 (Operators)

什麼是建立運算子？與可聯入通道的運算子不同，建立運算子一種函式，可用於根據一些常見預定義行為或聯合其它 Observable 來建立一個 Observable。

建立運算子的典型範例是 interval 函式。它將一個數字（而不是 Observable）作為輸入引數，併產生一個 Observable 作為輸出。

以下這個方法用subscribe去執行會在每秒(1000毫秒)輸出一一次從0開始。

```
import { interval } from 'rxjs';  
  
const observable = interval(1000 /* number of milliseconds */);
```

運算子（Operators）

運算子還有很多方法可以用可以參考以下網站。

<https://rxjs.angular.tw/guide/operators>

聯結建立運算子

這些是 Observable 的建立運算子，它們也具有聯結功能 —— 發出多個來源 Observable 的值。

- `combineLatest`
- `concat`
- `forkJoin`
- `merge`
- `partition`
- `race`
- `zip`

訂閱 (Subscription)

什麼是訂閱？訂閱是一個表示可釋放資源的物件，通常是 Observable 的一次執行。訂閱有一個重要的方法 `unsubscribe`，它不接受任何引數，只是釋放本訂閱所持有的資源。

```
import { interval } from 'rxjs';

const observable = interval(1000);
const subscription = observable.subscribe(x => console.log(x));
// Later:
// This cancels the ongoing Observable execution which
// was started by calling subscribe with an Observer.
subscription.unsubscribe();
```

訂閱 (Subscription)

多個訂閱也可以放在一起，以便呼叫一個訂閱的 `unsubscribe()` 就可以退訂多個訂閱。你可以透過將一個訂閱『新增』到另一個訂閱中來做到這一點。

```
import { interval } from 'rxjs';

const observable1 = interval(400);
const observable2 = interval(300);

const subscription = observable1.subscribe(x => console.log('first: ' + x));
const childSubscription = observable2.subscribe(x => console.log('second: ' + x));

subscription.add(childSubscription);

setTimeout(() => {
  // Unsubscribes BOTH subscription and childSubscription
  subscription.unsubscribe();
}, 1000);
```

主體 (Subjects)

什麼是主體？ RxJS Subject 是一種特殊型別的 Observable，它允許將值多播到多個 Observer。雖然普通的 Observable 是單播的（每個訂閱的 Observer 都擁有 Observable 的獨立執行），但 Subjects 是多播的。

```
import { Subject } from 'rxjs';

const subject = new Subject<number>();

subject.subscribe({
  next: (v) => console.log(`observerA: ${v}`),
});
subject.subscribe({
  next: (v) => console.log(`observerB: ${v}`),
});

subject.next(1);
subject.next(2);

// Logs:
// observerA: 1
// observerB: 1
// observerA: 2
// observerB: 2
```

结束

